

BT3040: Bioinformatics Assignment 1

Name: Sidharthan ; Roll No: BE21B039

Question 2:

The screenshot shows the REVSEQ web interface on the left and a terminal window on the right. The REVSEQ interface has an 'input section' with a text area containing the sequence 'CTCGGATTTGTAAGATCATGATCTCATACATAGTACCTAGCCATTG'. Below this is an 'output section' with a text area and a button 'Output Sequence Options'. At the bottom is an 'advanced section' with checkboxes for 'Reverse sequence', 'Complement sequence', and 'Tag output'. The terminal window on the right shows the command 'revseq06240548253270464454.rev cmd' and the output sequence 'GAGCCTAAACATTTCTAGTACTAGAGTATGTATCATGGATCGGTAAC'.

Question3

```
In [3]: # Define a function for finding complementary strand of given DNA strand
def complementary_strand_find(dna_strand):

    # create a dictionary for storing conversion pairs
    conv_dict = {"A" : "T", "T" : "A", "U" : "A", "G" : "C", "C" : "G"}
    # empty string define
    comp_strand = ""
    # looping through the given DNA strand one by one character at a time
    for base in dna_strand:

        # checking base is present in
        # dictionary or not
        # if not then error input
        # is given
        if base in conv_dict:

            # string concatenation
            comp_strand += conv_dict[base]

        else :
            print("The given sequence is incorrect")

            # assigning None to the variable
            # if wrong input is given
            comp_strand = None

            # break out of the loop
            # if wrong input is given
            break

    # return final result
    return comp_strand

# main code
# input the DNA strand
dna_strand = "CTCGGATTTGTAAGATCATGATCTCATACATAGTACCTAGCCATTG"

print("DNA strand is:",dna_strand)
#function call and printing the result
print("complementary strand is:",complementary_strand_find(dna_strand))

DNA strand is: CTCGGATTTGTAAGATCATGATCTCATACATAGTACCTAGCCATTG
complementary strand is: GAGCCTAAACATTTCTAGTACTAGAGTATGTATCATGGATCGGTAAC
```

Question 4

i)

The screenshot shows the TRANSEQ web interface. In the 'input section', the 'seqall' input is set to 'paste'. The 'Cut and Paste' area contains a DNA sequence. The 'output section' has an empty 'Output Sequence Name' field. The 'Execution mode' is set to 'interactive'. The 'GO' button is highlighted. To the right, a 'Saved Results' window titled 'transeq_Sun_Feb_18_12_33_18_IST_2024_30...' displays the translation of the first ORF (frame 1) as a protein sequence: DIVNSKKVHAMRKEQKRKQGRSMGSPMDYSPLPIDKHEPEFGPCRRKLDG.

ii) 4th ORF is where we can find the mentioned protein sequence

The screenshot shows the TRANSEQ web interface with the 'additional section' expanded. The 'Frame(s) to translate' is set to 'All six frames'. The 'Code to use' is set to 'Standard'. The 'Regions to translate' is set to 'Standard'. The 'GO' button is highlighted. To the right, a 'Saved Results' window titled 'transeq_Sun_Feb_18_12_33_53_IST_2024_79...' displays the translation of the first six ORFs (frames 1 to 6) as protein sequences. The 4th ORF (frame 4) is highlighted in blue and contains the protein sequence: PIQFSSAWTKFRMLVDGQRRVVHGRAHGALLALLPLLLLAHCMDFFTVHNV.

Question 5

```
In [1]: def translate(seq):

    table = {
        'ATA':'I', 'ATC':'I', 'ATT':'I', 'ATG':'M',
        'ACA':'T', 'ACC':'T', 'ACG':'T', 'ACT':'T',
        'AAC':'N', 'AAT':'N', 'AAA':'K', 'AAG':'K',
        'AGC':'S', 'AGT':'S', 'AGA':'R', 'AGG':'R',
        'CTA':'L', 'CTC':'L', 'CTG':'L', 'CTT':'L',
        'CCA':'P', 'CCC':'P', 'CCG':'P', 'CCT':'P',
        'CAC':'H', 'CAT':'H', 'CAA':'Q', 'CAG':'Q',
        'CGA':'R', 'CGC':'R', 'CGG':'R', 'CGT':'R',
        'GTA':'V', 'GTC':'V', 'GTG':'V', 'GTT':'V',
        'GCA':'A', 'GCC':'A', 'GCG':'A', 'GCT':'A',
        'GAC':'D', 'GAT':'D', 'GAA':'E', 'GAG':'E',
        'GGA':'G', 'GGC':'G', 'GGG':'G', 'GGT':'G',
        'TCA':'S', 'TCC':'S', 'TCG':'S', 'TCT':'S',
        'TTC':'F', 'TTT':'F', 'TTA':'L', 'TTG':'L',
        'TAC':'Y', 'TAT':'Y', 'TAA':'_', 'TAG':'_',
        'TGC':'C', 'TGT':'C', 'TGA':'_', 'TGG':'W',
    }

    protein = ""
    if len(seq)%3 == 0:
        for i in range(0, len(seq), 3):
            codon = seq[i:i + 3]
            protein+= table[codon]
    return protein

In [34]: translate('GACATTGTGAACAGTAAAAAGTCCATGCAATGCGCAAGGAGCAGAAGAGGAAGCAGGGCAAGCAGCGCTCCATGGGCTCTCCCATTGGACTACTCTCCTCTGCCCATCGACAAGCATC')

Out[34]: 'DIVNSKKVHAMRKEQRKQKGQRSMGSPMDYSLPIDKHEPEFGPCRRLDG'
```

Question 6

```
In [12]: seq="GACATGTGTGAACAGTAAAAAGTCCATGCAATGCGCAAGGAGCAGAAGAGCAAGGAGCGCTCTCCCATGGACTACTCTCTCTGCCCATCGACAAGCATGAGCCTC"
substrings=['AAG','GTC','GAG','ACTA','ATAT']
for sub in substrings:
    m=len(sub)
    n=len(seq)
    ctr=0
    pos=[]
    for j in range(n):
        if seq[j:j+m]==sub:
            ctr+=1
            pos.append(sub)
    print(sub)
    print('Position of substring= ')
    print(pos)
    print('Total number of occurrences =%d'%ctr)

AAG
Position of substring=
['AAG', 'AAG', 'AAG', 'AAG', 'AAG', 'AAG', 'AAG']
Total number of occurrences =7
GTC
Position of substring=
['GTC', 'GTC']
Total number of occurrences =2
GAG
Position of substring=
['GAG', 'GAG', 'GAG']
Total number of occurrences =3
ACTA
Position of substring=
['ACTA']
Total number of occurrences =1
ATAT
Position of substring=
[]
Total number of occurrences =0
```

Question 7

The DAN function calculates the melting temperature (T_m) of nucleic acids and can visually present the results if desired. On the other hand, the BANANA function creates plots illustrating bending and curvature data for typical DNA double helices. It examines the extent of local bending and overall curvature along a given B-DNA sequence, employing a selected bending model that determines

twist, roll, and tilt values based on the sequence. The software generates both graphical representations and a text file summarizing the results.

Question 8

```
In [32]: def energy(sequence):
          dict={'AA':-4,'AT':-7, 'AC':-5,'AG':-11,'TA':-7,'TT':-2,'TC':-3,'TG':-4,'CA':-9,'CT':-5,'CC':-6,'CG':-7,'GA':-9,'GT':-6,'GC':-5}
          energy=0
          n=len(sequence)
          n_bases=0
          pos=0
          for i in range(n-1):
              base=sequence[i:i+2]
              n_bases+=1
              energy+=dict[base]
              avg=energy/(n-1)
          return energy, avg

In [33]: energy('CTCGGATTTGTAAGATCATGATCTCATACATAGTACCTAGCCATTG')
Out[33]: (-289, -6.282608695652174)
```

Question 9

- i) The average melting point of ATATATATAT is 48.0022 degree Celsius

Your input seq is:

ATATATATAT

Physicochemical Properties:

Properties	Scaleunit	Average value
Stacking energy	kcal/mol	1.8
Enthalpy	kcal/mol	6.04444
Entropy	cal/mol/K	16.6222
Flexibility_shift	kJ mol ⁻¹ Å ⁻²	2.53
Flexibility_slide	kJ mol ⁻¹ Å ⁻²	9.66333
Free energy	kcal/mol	0.655556
Melting Temperature	degree	48.0022
Mobility to bend towards major groove	mu	1.09778
Mobility to bend towards minor groove	mu	1.03333
Probability contacting nucleosome core	%	6.75556
Rise stiffness	kcal/mol angstrom	7.80778
Roll stiffness	kcal/mol degree	19.3333
Shift stiffness	kcal/mol angstrom	0.892222
Slide stiffness	kcal/mol angstrom	2.66111
Tilt stiffness	kcal/mol degree	28
Twist stiffness	kcal/mol degree	25.8889

- ii) The average melting temperature is 107.867 degree Celsius

Your input seq is:

GC GCGCGCGC

Physicochemical Properties:

Properties	Scaleunit	Average value
Stacking energy	kcal/mol	1.75556
Enthalpy	kcal/mol	11.0778
Entropy	cal/mol/K	27.5556
Flexibility_shift	kJ mol ⁻¹ Å ⁻²	6.49111
Flexibility_slide	kJ mol ⁻¹ Å ⁻²	4.19778
Free energy	kcal/mol	1.85889
Melting Temperature	degree	107.867
Mobility to bend towards major groove	mu	0.997778
Mobility to bend towards minor groove	mu	1.20556
Probability contacting nucleosome core	%	3.37778
Rise stiffness	kcal/mol angstrom	8.06333
Roll stiffness	kcal/mol degree	21.5556
Shift stiffness	kcal/mol angstrom	1.14667
Slide stiffness	kcal/mol angstrom	2.33889
Tilt stiffness	kcal/mol degree	31.5556
Twist stiffness	kcal/mol degree	20.1111

Question 10

AT content – 58.333333%

GV content – 41.666667%

Your input seq is:

AAATGGCCCTAA

Nucleotide Content:

	Nucleotide content in %
AT_content	58.333333
Adenine_content	41.666667
Cytosine_content	25.000000
GC_content	41.666667
Guanine_content	16.666667
Keto_GT_content	33.333333
Purine_AG_content	58.333333
Thymine_content	16.666667